



ELKATECH LAUNCHPAD

Technical Requirements Document

ElkaTech Launchpad — Service Platform

Document: Technical Requirements Document

Project: ElkaTech Launchpad

Version: v2.0

Date: 11 June 2026

Status: Current — feature/owner-support-roles

Table of Contents

1. 1. System Architecture

2. 2. Frontend Architecture

3. 3. Backend / Gateway Architecture

4. 4. Internal Services Architecture

5. 5. Auth / Session Model

6. 6. RBAC Enforcement

7. 7. API Design

8. 8. Request Workflow Backend

9. 9. Customer Machine Backend Model

10. 10. Attachment Upload / R2 Design

11. 11. Notification / Outbox Design

12. 12. Database / Postgres Usage

13. 13. Shared Contracts / Zod Validation

14. 14. Security Requirements

15. 15. Deployment / Vercel Requirements

16. 16. Environment Variables

17. 17. api/*.mjs Bundling Requirement

18. 18. Error Handling / Logging

19. 19. Testing Strategy

20. 20. Operational Risks

1. System Architecture

ElkaTech Launchpad is an npm-workspaces monorepo. A client-side React/Vite app calls a single Fastify **gateway** at `/api/*`. The gateway owns browser-facing concerns — session cookies, CSRF, rate limiting, and role enforcement — and calls four internal Fastify services using a shared `x-internal-token`. Auth and service-desk write outbox events that the notification service polls into email.

```
Browser → React/Vite app → Gateway (/api/*)
Gateway → auth | catalog | service-desk      (x-internal-token)
service-desk → Cloudflare R2 (presigned)
auth/service-desk → Postgres outbox → notification → SMTP
auth, catalog, service-desk, notification → PostgreSQL (per-schema)
```

In production every service is bundled into `api/*.mjs` and deployed as Vercel serverless functions; each deployment routes internal traffic back to itself.

2. Frontend Architecture

- React 18 + TypeScript + Vite, React Router, TanStack Query, Tailwind, and shadcn/Radix UI primitives.
- A single `ThemeProvider` (light/dark/system, persisted) and `BrowserRouter`.
- `apps/web/src/lib/api.ts` is the API client: it attaches `x-csrf-token` from the CSRF cookie on mutations and always sends credentials.
- `ProtectedRoute` gates portal routes by session and role; `PortalShell` renders the role-aware sidebar.
- Public category pages render from local component data; the portal reads the catalog through the gateway.

3. Backend / Gateway Architecture

- The gateway is the only public boundary. Every `/api/*` route resolves the session via the auth service, checks CSRF on mutations, applies rate limits on sensitive endpoints, and enforces role permissions before forwarding.
- Permission decisions call shared helpers from `@elkatech/contracts` (`canDeleteUsers`, `canApproveUsers`, `canAssignRequests`, `canCreateRequestForCustomer`, `assignableRolesFor`, `canManageTargetUser`, ...) so server enforcement matches the UI.
- Internal calls forward the actor identity via `x-user-*` headers plus the shared `x-internal-token`.

4. Internal Services Architecture

SERVICE	RESPONSIBILITY	SCHEMA
auth	Accounts, sessions, Firebase/Google linking, verify/reset/invite tokens, approval, profiles, roles	auth
catalog	Category/product reads, optional Redis cache	catalog
service-desk	Request lifecycle, messages, history, assignment, machines, attachments	service_desk
notification	Polls outbox tables, sends SMTP email, records deliveries	notification

All non-health internal endpoints require `x-internal-token`. Services share one Postgres connection helper, so cross-schema operations (for example a hard delete that spans `auth` and `service_desk`) can run transactionally.

5. Auth / Session Model

- Sign-in obtains a Firebase ID token, exchanged at `POST /api/auth/firebase/session`; the Admin SDK verifies it, then the auth service creates or links the local user and issues a session.
- A session row stores a hashed token and a CSRF token; the gateway sets the session cookie (httpOnly) and a readable CSRF cookie.
- `GET /api/auth/me` resolves the session to a user including `role`, `approvalStatus`, `accountOrigin`, and `profileCompleted`.
- A legacy email/password path (`/api/auth/login`, reset, verify) remains for pre-Firebase accounts.

6. RBAC Enforcement

- Five roles: `customer`, `engineer`, `support`, `owner`, `admin`.
- The gateway gates each route by allowed roles and re-checks fine-grained rules (e.g. owner may assign non-admin roles only and may never modify an admin account; delete is admin-only).
- The service-desk independently re-checks the actor role from `x-user-role` (defence in depth) for assignment and machine management.
- The same helper functions power UI visibility, so a hidden control always maps to a backend `403`.

7. API Design

- REST-style JSON over the gateway under `/api`. Mutations require a valid `x-csrf-token`. Request and response shapes are validated with Zod contracts.

- Representative endpoint groups: `auth`, `catalog`, `me` (self profile, machines), `requests` (+ messages, claim, assign, status, cancel, attachments), `admin` (users, approvals, roles, machines, health), and staff dashboards (`customer-activity`, `support/summary`, `staff/customers`, `engineers`).
- Errors return structured `{ message }` (and sometimes a `code`); the gateway forwards 4xx business errors verbatim but masks internal 5xx detail.

8. Request Workflow Backend

- Creation validates approval, email verification, and (for customers) profile completion and machine ownership; product/site/serial/phone are copied from the selected machine.
- Status transitions are enforced by a transition map in the service-desk workflow module; assignment sets `assigned` and records assigned/reassigned history.
- Listing is role-scoped: customers see their own requests; engineers see assigned/unassigned; support/owner/admin see all; an optional `scope=queue` and `statusGroup` filter the view.

9. Customer Machine Backend Model

- `service_desk.customer_machines` holds one row per owned unit, admin/owner controlled, with a product snapshot, labels, site, dates, an admin-only internal serial, and `active` / `inactive` status.
- `customer_id` is the `auth.users` id stored as a bare uuid (no cross-schema FK), consistent with the service boundary; ownership is enforced in the application layer.
- The customer-facing machine view never exposes the internal serial or notes.

10. Attachment Upload / R2 Design

- `POST /api/requests/:id/attachments/presign` issues a short-lived S3-compatible presigned `PUT` scoped to a request-prefixed object key, after validating type/size; returns `501` when R2 is unconfigured.
- The browser `PUT` s bytes directly to R2; then `POST /api/requests/:id/attachments` persists metadata and records an `attachment_added` history event.
- Read URLs derive from `R2_PUBLIC_BASE_URL` when set, else a presigned `GET` ; the URL is never persisted, so rotating the bucket needs no data migration.

11. Notification / Outbox Design

- Auth and service-desk write domain events to `auth.outbox` and `service_desk.outbox` within their write transactions.
- The notification service polls both tables, sends SMTP email, and records a row in `notification.deliveries` per `(event_id, recipient_email)` for idempotent retries.

- On serverless, a fire-and-forget poke triggers outbox draining since long-lived intervals do not run between requests.

12. Database / Postgres Usage

- One PostgreSQL database with four schemas: `auth`, `catalog`, `service_desk`, `notification`.
- Migrations live under `services//migrations/.sql` and are applied by `npm run db:migrate`, tracked idempotently in `public.platform_migrations`.
- Local development uses Docker Postgres; production uses Neon (pooled URL via `POSTGRES_URL`).

13. Shared Contracts / Zod Validation

- `packages/contracts` exports Zod schemas, inferred types, role/permission helpers, issue-type labels, and catalog seed data.
- Both the frontend and the services import these, so request/response shapes and permission rules cannot drift between client and server.

14. Security Requirements

- HttpOnly session cookie + readable CSRF cookie; CSRF verified on mutations.
- Shared `x-internal-token` gates all internal endpoints.
- Server-side RBAC with target-aware checks; admin-only permanent deletion; protected system/bootstrap admin and self-modification guards.
- Private-write object storage (presigned uploads only); rate limiting on auth-sensitive endpoints.

15. Deployment / Vercel Requirements

- `vercel.json` sets the Vite build, output `apps/web/dist`, and rewrites `/api/` to `api/` `.mjs` plus SPA fallback to `index.html`.
- `packages/config/src/env.ts` rewrites service URLs when `VERCEL=1` so each deployment calls itself; previews use `VERCEL_URL`, production uses `VERCEL_PROJECT_PRODUCTION_URL`.
- Disable Vercel Deployment Protection for previews (or restrict to production) so internal server-to-server calls succeed.

16. Environment Variables

VARIABLE	PURPOSE
<code>POSTGRES_URL</code> / <code>DATABASE_URL</code>	Postgres connection (hosted / local)

VARIABLE	PURPOSE
<code>INTERNAL_SERVICE_TOKEN</code>	Shared gateway↔services token
<code>APP_BASE_URL</code> , <code>*_SERVICE_URL</code> , <code>GATEWAY_URL</code>	Service routing and email links
<code>SESSION_COOKIE_NAME</code> , <code>CSRF_COOKIE_NAME</code> , <code>SESSION_TTL_HOURS</code>	Session config
<code>SMTP_HOST/PORT/USER/PASS/FROM</code>	Email delivery
<code>FIREBASE_PROJECT_ID/CLIENT_EMAIL/PRIVATE_KEY</code>	Firebase Admin SDK
<code>VITE_FIREBASE_*</code>	Firebase web client config
<code>R2_ACCOUNT_ID</code> , <code>R2_ACCESS_KEY_ID</code> , <code>R2_SECRET_ACCESS_KEY</code>	R2 credentials
<code>R2_BUCKET_NAME</code> , <code>R2_ENDPOINT</code> , <code>R2_PUBLIC_BASE_URL</code>	R2 bucket / endpoints
<code>MAX_REQUEST_ATTACHMENT_MB</code> , <code>ALLOWED_REQUEST_ATTACHMENT_TYPES</code>	Upload limits
<code>BOOTSTRAP_ADMIN_EMAIL</code> , <code>BOOTSTRAP_ADMIN_PASSWORD</code>	Initial admin

`packages/config/src/env.ts` is the authoritative schema; `.env.example` is the local template. The R2 endpoint must be the S3 API host (no bucket name); the public base URL is the read host.

17. api/*.mjs Bundling Requirement

- `scripts/bundle-api.cjs` (esbuild) bundles each service entry point into a self-contained `api/<service>.mjs`. Vercel runs the **committed** bundles and does not rebundle.
- After changing anything under `services/` or `packages/`, re-run `node scripts/bundle-api.cjs` and commit the regenerated bundles, or production runs stale backend code.

18. Error Handling / Logging

- Services use Fastify's logger. The gateway forwards business 4xx errors but returns a generic 502 for internal 5xx, logging the underlying error.
- Validation failures surface as 4xx from Zod parsing. Auth/role failures return 401/403 with a clear message.

19. Testing Strategy

- Vitest unit tests cover gateway approval/RBAC logic and service-desk workflow transitions.

- Type safety is enforced with `tsc --noEmit` per workspace; the web build runs through Vite.
- Recommended pre-merge checks: per-service typecheck, `npm run build -w @elkatech/web`, and `npm run test`.

20. Operational Risks

- **Branch-per-deployment databases:** if enabled, preview deployments get an un-migrated database; keep it disabled or migrate the branch explicitly.
- **Stale bundles:** forgetting to re-run `bundle-api.cjs` ships old backend logic.
- **R2 CORS:** missing/incorrect CORS blocks browser uploads despite valid presigned URLs.
- **Deployment protection:** SSO protection on previews breaks internal routing.
- **Node version drift:** tooling requires Node 22; older Node breaks the dev and migration scripts.